Apache Kafka — Guida alle Configurazioni di Producer e Consumer (NSDS)

1. Matrice Rapida: Quali Configurazioni Impostare su Producer e Consumer

Scenario / Garanzia Richiesta	Configurazioni `ProducerConfig` (`Properties`)	Configurazioni `ConsumerConfig` (`Properties`)	Metodi Chiave nel Codice

1. Base (Default / Fire-and-Forget + Auto-Commit)	`BOOTSTRAP_SERVERS_CONFIG = "localhost:9092"`
	`KEY_SERIALIZER_CLASS_CONFIG = ...`
	`VALUE_SERIALIZER_CLASS_CONFIG = ...`
		`BOOTSTRAP_SERVERS_CONFIG = "localhost:9092"`
		`GROUP_ID_CONFIG = "groupA"`
		`ENABLE_AUTO_COMMIT_CONFIG = "true"`
		`AUTO_COMMIT_INTERVAL_MS_CONFIG = "15000"`
		`AUTO_OFFSET_RESET_CONFIG = "latest"`
		`KEY_DESERIALIZER_CLASS_CONFIG = ...`
		`VALUE_DESERIALIZER_CLASS_CONFIG = ...`
			Producer: `producer.send(record);` (asincrono)
			Consumer: `consumer.poll( ... )` (gli offset sono salvati in automatico ogni 15s)

2. Producer Sincrono (Attesa ACK dal Broker)	Stesse configurazioni del Producer Base	Qualsiasi Consumer	`Future<RecordMetadata> f = producer.send(record);` `RecordMetadata ack = f.get();` (legge `topic()` , `partition()` , `offset()`)
3. Producer Idempotente (No duplicati da retry tra Producer e Partizione)	Base + `ENABLE_IDEMPOTENCE_CONFIG = "true"`	Qualsiasi Consumer	Il producer numera i messaggi (*sequence numbers*); il broker scarta automaticamente i duplicati dovuti a ritrasmissioni
4. Consumer con Commit Manuale (*At-Least-Once*)	Qualsiasi Producer	Base ma con: `ENABLE_AUTO_COMMIT_CONFIG = "false"`	Chiamare `consumer.commitSync()` **dopo** aver elaborato il batch o ogni *N* messaggi (`commitEvery`)
5. Scrittura e Lettura Transazionale (Tutto-o-Niente)	Base + `TRANSACTIONAL_ID_CONFIG = "myTxId"` *(implica `ENABLE_IDEMPOTENCE_CONFIG = "true"`)*	Base + `ISOLATION_LEVEL_CONFIG = "read_committed"` *(scarta i messaggi delle transazioni abortite)*	**Producer**: `initTransactions()` (1 volta) → `beginTransaction()` → `send( ... )` → `commitTransaction()` (o `flush()` + `abortTransaction()`)

6. Atomic Forwarder (*End-to-End Exactly-Once* Read–Process–Write)	Base +
	`TRANSACTIONAL_ID_CONFIG = "forwarderTxId"`
	`ENABLE_IDEMPOTENCE_CONFIG = "true"`
		Base +
		`ENABLE_AUTO_COMMIT_CONFIG = "false"`
		`ISOLATION_LEVEL_CONFIG = "read_committed"`
			Il consumer **non** usa `commitSync()` . Il producer committa sia i messaggi prodotti sia gli offset del consumer (`lastOffset + 1`) con:
			`producer.sendOffsetsToTransaction(map, consumer.groupMetadata());`
			`producer.commitTransaction();`

2. Configurazioni del PRODUCER (`ProducerConfig`) — Analisi Dato per Dato

2.1 Tabella di Tutte le Proprietà del Producer

Costante `ProducerConfig.*`	Chiave Stringa	Valore da Passare (`String`)	Default	Effetto sul Comportamento del Producer
`BOOTSTRAP_SERVERS_CONFIG`	`"bootstrap.servers"`	`"localhost:9092"`	*Obbligatorio*	Indirizzo iniziale di uno o più broker Kafka per connettersi al cluster e scaricare i metadati delle partizioni.
`KEY_SERIALIZER_CLASS_CONFIG`	`"key.serializer"`	`StringSerializer.class.getName()`	*Obbligatorio*	Specifica come convertire la chiave `K` del `ProducerRecord<K, V>` in `byte[]` .
`VALUE_SERIALIZER_CLASS_CONFIG`	`"value.serializer"`	`StringSerializer.class.getName()`	*Obbligatorio*	Specifica come convertire il valore `V` del `ProducerRecord<K, V>` in `byte[]` .
`ENABLE_IDEMPOTENCE_CONFIG`	`"enable.idempotence"`	`String.valueOf(true)` oppure `String.valueOf(false)`	`"false"`	**Se `"true"`** : garantisce **Exactly-Once tra il producer e la partizione**. Evita che un messaggio venga scritto due volte se l'ACK si perde e il producer lo ritrasmette.
`TRANSACTIONAL_ID_CONFIG`	`"transactional.id"`	`"myTransactionalId"` *(Stringa univoca per ogni istanza di producer)*	`null`	**Abilita le transazioni**. Registra il producer presso il *Transaction Coordinator* del broker per eseguire scritture atomiche su più partizioni/topic e isolare istanze duplicate (zombie fencing).
`ACKS_CONFIG` *(opzionale)*	`"acks"`	`"0"` , `"1"` , `"all"`	`"1"` (`"all"` se idempotente)	Numero di repliche che devono confermare la scrittura: `"0"` (nessun ACK), `"1"` (solo la replica leader), `"all"` (leader + tutte le repliche follower in-sync).

2.2 Approfondimento: Cosa Cambia Togliendo o Aggiungendo le Configurazioni del Producer

A) Chiave del Messaggio (`key`) e Scelta della Partizione in `ProducerRecord<K, V>`

Il producer decide su quale partizione del topic scrivere il messaggio in base a come costruisci `ProducerRecord`:

- `new ProducerRecord<>(topic, key, value)` *(con chiave specificata)*:
- Il partitioner di default calcola un **hash della chiave** (`hash(key) % numPartitions`).
- **Garanzia fondamentale**: tutti i messaggi con la **stessa** `key` finiscono **sempre nella stessa partizione** e vengono quindi salvati e letti in **ordine FIFO rigoroso**.
- `new ProducerRecord<>(topic, value)` *(senza chiave, cioè `key = null`)*:
- I messaggi vengono distribuiti tra le partizioni disponibili per bilanciare il carico, ma **non c'è alcuna garanzia di località** né di ordine relativo tra messaggi diversi.
- `new ProducerRecord<>(topic, partitionIndex, key, value)` *(con partizione forzata)*:
- Scavalca l'hash della chiave e scrive direttamente nella partizione `partitionIndex` (`0, 1, ...`).

B) Invio Asincrono vs. Invio Sincrono (`waitAck`)

`producer.send(record)` è **sempre asincrono** di base e restituisce immediatamente un `Future<RecordMetadata>`:

- `waitAck = false` **(Asincrono)**: chiami solo `producer.send(record);` e prosegui subito col messaggio successivo (massimo throughput).
- `waitAck = true` **(Sincrono / Bloccante)**: chiami `RecordMetadata ack = producer.send(record).get();`. Il thread del producer si blocca finché il broker non conferma la scrittura e ti restituisce i dati in `ack`:
- `ack.topic()` → nome del topic (`String`)
- `ack.partition()` → numero della partizione in cui è stato scritto (`int`)
- `ack.offset()` → posizione sequenziale assegnata nel log della partizione (`long`)

C) `ENABLE_IDEMPOTENCE_CONFIG = "true"` (Producer Idempotente)

- **Problema senza idempotenza (`"false"`)**: il producer invia un messaggio `M1` al broker; il broker lo scrive nella partizione ma l'ACK si perde sulla rete. Scatta il timeout e il producer rinvia `M1` : il broker lo scrive una seconda volta (*At-Least-Once* → **messaggio duplicato**).
- **Soluzione con** `ENABLE_IDEMPOTENCE_CONFIG = "true"` : il broker assegna al producer un ID univoco (`PID`) e il producer allega un **numero di sequenza crescente** a ogni messaggio inviato verso una partizione. Se il broker riceve di nuovo `M1` con un sequence number già visto, **scarta il duplicato** e risponde semplicemente con l'ACK (*Exactly-Once* sulla singola partizione).

D) `TRANSACTIONAL_ID_CONFIG` (Producer Transazionale: `commit` vs `abort` + `flush`)

- **Quando serve**: quando vuoi scrivere **più messaggi** (anche su topic e partizioni diverse) **in modo atomico** (*o vengono scritti tutti, o nessuno*), oppure quando vuoi fare un *Atomic Forwarder* (scrivere l'output e committare gli offset del consumer nella stessa transazione).
- **Regole obbligatorie nel codice**:
- Impostare `props.put(ProducerConfig.TRANSACTIONAL_ID_CONFIG, "myTransactionalId");` (e opzionalmente `ENABLE_IDEMPOTENCE_CONFIG = "true"`, che viene comunque attivato di default dalle transazioni).
- Chiamare `producer.initTransactions();` **una sola volta** prima del ciclo `for` / `while` .
- Per ogni transazione:
  - Aprire con `producer.beginTransaction();`
  - Inviare uno o più record con `producer.send(record);`
  - Chiudere con `producer.commitTransaction();` (conferma) oppure `producer.abortTransaction();` (annulla).
- **Perché in `TransactionalProducer.java` si chiama `producer.flush()` prima di `producer.abortTransaction()` ?**:
- `producer.send(record)` mette i messaggi in un buffer locale in RAM del producer.
- Se chiami subito `abortTransaction()` senza `flush()` , i messaggi vengono semplicemente cancellati dal buffer locale del producer senza mai arrivare al broker.
- Chiamando `producer.flush()` prima di `abortTransaction()` , forzi l'invio fisico dei messaggi sul disco del broker, seguiti subito dal marker di transazione abortita. Così puoi vedere che un consumer `"read_uncommitted"` li legge comunque, mentre un consumer `"read_committed"` li scarta!

---

3. Configurazioni del CONSUMER (`ConsumerConfig`) — Analisi Dato per Dato

3.1 Tabella di Tutte le Proprietà del Consumer

Costante `ConsumerConfig.*`	Chiave Stringa	Valore da Passare (`String`)	Default	Effetto sul Comportamento del Consumer
`BOOTSTRAP_SERVERS_CONFIG`	`"bootstrap.servers"`	`"localhost:9092"`	*Obbligatorio*	Indirizzo iniziale del/dei broker Kafka a cui connettersi.
`GROUP_ID_CONFIG`	`"group.id"`	`"groupA"` , `"groupB"` , ecc.	*Obbligatorio*	Identifica il **Consumer Group** a cui appartiene il consumer. Controlla la ripartizione delle partizioni e il salvataggio degli offset.
`KEY_DESERIALIZER_CLASS_CONFIG`	`"key.deserializer"`	`StringDeserializer.class.getName()`	*Obbligatorio*	Converte i `byte[]` della chiave letta dal broker nel tipo Java `K` (deve combaciare col `KEY_SERIALIZER` del producer).
`VALUE_DESERIALIZER_CLASS_CONFIG`	`"value.deserializer"`	`StringDeserializer.class.getName()`	*Obbligatorio*	Converte i `byte[]` del valore letto dal broker nel tipo Java `V` (deve combaciare col `VALUE_SERIALIZER` del producer).
`ENABLE_AUTO_COMMIT_CONFIG`	`"enable.auto.commit"`	`String.valueOf(true)` oppure `String.valueOf(false)`	`"true"`	**Se `"true"`** : il consumer salva gli offset da solo a intervalli regolari. **Se `"false"`** : il salvataggio

Costante `ConsumerConfig.*`	Chiave Stringa	Valore da Passare (`String`)	Default	Effetto sul Comportamento del Consumer
				automatico è spento; devi committare tu con `consumer.commitSync()` o tramite transazione del producer.
`AUTO_COMMIT_INTERVAL_MS_CONFIG`	`"auto.commit.interval.ms"`	`"15000"` (15s), `"5000"` (5s)	`"5000"`	Ogni quanti millisecondi il consumer salva in background l'offset dell'ultimo `poll()` (attivo solo se `enable.auto.commit = "true"`).
`AUTO_OFFSET_RESET_CONFIG`	`"auto.offset.reset"`	`"latest"` oppure `"earliest"`	`"latest"`	Stabilisce da dove partire **SOLO SE non esiste alcun offset già salvato** per quel `group.id` : - `"latest"` : legge solo i **nuovi** messaggi prodotti dopo l'avvio. - `"earliest"` : legge **tutti** i messaggi fin dall'offset `0` .
`ISOLATION_LEVEL_CONFIG`	`"isolation.level"`	`"read_committed"` oppure `"read_uncommitted"`	`"read_uncommitted"`	Stabilisce cosa leggere se il producer usa transazioni: - `"read_uncommitted"` (default): legge tutto, **anche i messaggi abortiti**. - `"read_committed"` : legge **solo** i messaggi di transazioni committate (e quelli non transazionali).

3.2 Approfondimento: Come Funzionano le Configurazioni del Consumer

A) `GROUP_ID_CONFIG` : Stesso Gruppo (Load Balancing) vs. Gruppi Diversi (Broadcast)

Ogni Consumer Group rappresenta **un singolo consumer logico**. Dato un topic con ***P* partizioni**:

Configurazione di `GROUP_ID_CONFIG`	Come vengono assegnate le *P* partizioni?	Chi riceve i messaggi del topic?
1 solo Consumer con `group.id = "groupA"`	Tutte le *P* partizioni sono assegnate a quell'unico consumer.	Riceve tutti i messaggi di tutte le partizioni.
***C* ≤ *P* Consumer** con lo **STESSO** `group.id = "groupA"`	Le *P* partizioni vengono **divise (partizionate)** tra i *C* consumer (una partizione è assegnata a **un solo** consumer del gruppo).	**Load Balancing**: ogni messaggio viene letto da **uno solo** dei consumer del gruppo (elaborazione parallela).
***C* > *P* Consumer** con lo **STESSO** `group.id = "groupA"`	*P* consumer ricevono *1* partizione ciascuno; i restanti ***C* - *P* consumer restano fermi (inattivi / idle)**.	**Limite di scalabilità**: non ha mai senso avere più consumer nello stesso gruppo del numero di partizioni *P* del topic!
Più Consumer con **DIVERSO** `group.id` (es. `"groupA"` e `"groupB"`)	Ogni gruppo riceve tutte le *P* partizioni in modo indipendente e mantiene il proprio offset separato.	**Broadcast (Pub/Sub)**: ogni messaggio viene consegnato a **ciascun** Consumer Group iscritto al topic.

B) `AUTO_OFFSET_RESET_CONFIG` (`"latest"` vs `"earliest"`) e Interazione con `GROUP_ID_CONFIG`

Attenzione (errore classico d'esame/lab): `AUTO_OFFSET_RESET_CONFIG` entra in gioco **esclusivamente quando per il** `GROUP_ID_CONFIG` **scelto non esiste ancora nessun offset salvato** nel topic `__consumer_offsets` di Kafka!

Stato del `GROUP_ID_CONFIG` sul Broker	Valore di `AUTO_OFFSET_RESET_CONFIG`	Cosa fa il Consumer all'avvio?
Gruppo nuovo (primo avvio assoluto di `"groupA"`)	`"latest"` *(default)*	Si posiziona alla **fine** del log: ignora tutti i messaggi già presenti e aspetta solo quelli nuovi inviati da questo momento in poi.
Gruppo nuovo (primo avvio assoluto di `"groupA"`)	`"earliest"`	Si posiziona all'**inizio** del log (`offset = 0`): scarica e legge tutta la storia dei messaggi già presenti nel topic.
Gruppo già esistente (`"groupA"` ha già fatto commit in passato)	`"latest"` **oppure** `"earliest"`	**Ignora** `AUTO_OFFSET_RESET_CONFIG` **!** Riparte esattamente dall'**ultimo offset committato** da `"groupA"` prima di spegnersi. *(Se vuoi rileggere tutto da zero con `"earliest"` , devi cambiare il nome di `GROUP_ID_CONFIG` , es. `"groupB"` !)*

C) `ENABLE_AUTO_COMMIT_CONFIG` : I 3 Modi di Gestire il Commit degli Offset

1. **Modo 1 — Auto-Commit Automatico (`ENABLE_AUTO_COMMIT_CONFIG = "true"` , default)**:
   - Imposti anche `AUTO_COMMIT_INTERVAL_MS_CONFIG = "15000"` (15 secondi).
   - Il consumer salva automaticamente gli offset ogni 15s durante le chiamate a `poll()` .
   - *Problema*: se il consumer crasha a 14 secondi dopo aver già stampato/elaborato dei messaggi ma prima dell'auto-commit, al riavvio **rileggerà gli stessi messaggi** (duplicati). Viceversa, se l'auto-commit scatta appena ricevuti i record da `poll()` ma il consumer crasha a metà del ciclo `for` prima di finirne l'elaborazione, quei messaggi **non verranno più riletti** (perdita).
2. **Modo 2 — Commit Manuale dal Consumer (`ENABLE_AUTO_COMMIT_CONFIG = "false"` + `consumer.commitSync()`)**:
   - Il consumer non salva mai gli offset da solo.

- Nel ciclo di lettura, **dopo** aver elaborato il messaggio (o ogni `commitEvery = 100` messaggi, oppure alla fine del ciclo `for` di `poll()`), chiami esplicitamente `consumer.commitSync();`.
- Garantisce **At-Least-Once**: nessun messaggio viene mai committato prima di essere stato elaborato.

3. **Modo 3 — Commit Transazionale tramite il Producer (`ENABLE_AUTO_COMMIT_CONFIG = "false"` + `producer.sendOffsetsToTransaction( ... )`)**:
   - Usato nell'**Atomic Forwarder** (quando un componente è sia Consumer di `inputTopic` sia Producer verso `outputTopic`).
   - Il consumer ha `ENABLE_AUTO_COMMIT_CONFIG = "false"`, ma **non chiama mai** `consumer.commitSync()`.
   - È il **producer** che, dentro la sua transazione, calcola per ogni `TopicPartition` l'ultimo offset letto (`lastOffset`) e invia `new OffsetAndMetadata(lastOffset + 1)` al *Transaction Coordinator* tramite `producer.sendOffsetsToTransaction(map, consumer.groupMetadata());` seguito da `producer.commitTransaction();`.
   - **Perché `lastOffset + 1`?**: perché l'offset salvato in Kafka indica sempre **la prossima casella da leggere**. Se l'ultimo messaggio letto ha offset `9`, devi salvare `10` affinché la prossima lettura parta dal messaggio `10`!

D) `ISOLATION_LEVEL_CONFIG`: `"read_committed"` vs. `"read_uncommitted"`

Quando un `TransactionalProducer` scrive su un topic, i suoi messaggi possono essere **committati** (`commitTransaction()`) oppure **abortiti** (`abortTransaction()`).

- `ISOLATION_LEVEL_CONFIG = "read_uncommitted"` *(Attenzione: è il **DEFAULT** di Kafka!)*:
- Il consumer legge **tutti** i messaggi presenti fisicamente nella partizione, **compresi quelli di transazioni ancora aperte o abortite** (se erano stati inviati al broker tramite `flush()`).
- `ISOLATION_LEVEL_CONFIG = "read_committed"`:
- Il consumer filtra e restituisce **solo** i messaggi appartenenti a transazioni che hanno eseguito `commitTransaction()` con successo (oltre ai normali messaggi inviati da producer non transazionali).
- **Va sempre impostato a `"read_committed"`** in tutti gli esercizi che parlano di transazioni, *Atomic Forwarder* o *Exactly-Once Semantics (EOS)*.

3.3 Corrispondenza tra Tipi `<K, V>` e Classi Serializer / Deserializer

I tipi generici di `KafkaProducer<K, V>` e `KafkaConsumer<K, V>` devono corrispondere alle classi impostate nelle `Properties` (package `org.apache.kafka.common.serialization.*`):

Tipo di Dato Java (`K` o `V`)	`ProducerConfig` (`KEY_SERIALIZER_CLASS_CONFIG` / `VALUE_SERIALIZER_CLASS_CONFIG`)	`ConsumerConfig` (`KEY_DESERIALIZER_CLASS_CONFIG` / `VALUE_DESERIALIZER_CLASS_CONFIG`)
`String`	`StringSerializer.class.getName()`	`StringDeserializer.class.getName()`
`Integer`	`IntegerSerializer.class.getName()`	`IntegerDeserializer.class.getName()`
`Long`	`LongSerializer.class.getName()`	`LongDeserializer.class.getName()`
`Double`	`DoubleSerializer.class.getName()`	`DoubleDeserializer.class.getName()`
`byte[]`	`ByteArraySerializer.class.getName()`	`ByteArrayDeserializer.class.getName()`

---

4. Tutti i Template di Configurazione Pronti all'Uso (Codice Completo)

4.1 Producer Base (Asincrono e Sincrono con `waitAck`) + Producer Idempotente

(Rif: `BasicProducer.java`, `IdempotentProducer.java`)

```
// --- CONFIGURAZIONE PRODUCER (BASE O IDEMPOTENTE) ---
final Properties props = new Properties();
props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class.getName());
props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, StringSerializer.class.getName());

// Decommenta la riga seguente per avere un PRODUCER IDEMPOTENTE (Exactly-Once tra producer e partizione):
// props.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, String.valueOf(true));

final KafkaProducer<String, String> producer = new KafkaProducer<>(props);

// Creazione e invio del record (stessa key → stessa partizione garantita)
final ProducerRecord<String, String> record = new ProducerRecord<>("topicA", "Key1", "Val1");
final Future<RecordMetadata> future = producer.send(record);

// Opzionale: se waitAck == true, attende la conferma sincrona dal broker e legge partizione e offset
if (waitAck) {
    try {
        RecordMetadata ack = future.get();
        System.out.println("Ack for topic " + ack.topic()
                + ", partition " + ack.partition()
                + ", offset " + ack.offset());
    } catch (InterruptedException | ExecutionException e) {
        e.printStackTrace();
    }
}

producer.close();
```

4.2 Consumer Base (Auto-Commit) vs. Consumer con Commit Manuale (`commitSync`)

(Rif: `BasicConsumer.java`, `BasicConsumerManual.java`)

```
// --- CONFIGURAZIONE CONSUMER (AUTO-COMMIT VS COMMIT MANUALE) ---
final boolean autoCommit = false;          // true = Auto-Commit, false = Commit Manuale con commitSync()
final int autoCommitIntervalMs = 15000;    // Usato solo se autoCommit == true
final int commitEvery = 100;               // Usato solo se autoCommit == false
final String offsetResetStrategy = "latest"; // "latest" (solo nuovi) oppure "earliest" (da offset 0 se gruppo nuovo)

final Properties props = new Properties();
props.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
props.put(ConsumerConfig.GROUP_ID_CONFIG, "groupA");
props.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, String.valueOf(autoCommit));
if (autoCommit) {
    props.put(ConsumerConfig.AUTO_COMMIT_INTERVAL_MS_CONFIG, String.valueOf(autoCommitIntervalMs));
}
props.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, offsetResetStrategy);
props.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());
props.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());

KafkaConsumer<String, String> consumer = new KafkaConsumer<>(props);
consumer.subscribe(Collections.singletonList("topicA"));

int numConsumed = 0;
while (true) {
    final ConsumerRecords<String, String> records = consumer.poll(Duration.of(5, ChronoUnit.MINUTES));
    for (final ConsumerRecord<String, String> record : records) {
        System.out.println("Partition: " + record.partition()
                + "\tOffset: " + record.offset()
                + "\tKey: " + record.key()
                + "\tValue: " + record.value());

        if (!autoCommit) {
            numConsumed++;
            if (numConsumed == commitEvery) {
                numConsumed = 0;
                consumer.commitSync(); // Salva manualmente gli offset sul broker ogni 100 messaggi
            }
        }
    }
}
```

4.3 Producer Transazionale e Consumer Transazionale (`read_committed` vs `read_uncommitted`)

(Rif: `TransactionalProducer.java`, `TransactionalConsumer.java`)

```
// ========================================================================
// 1. TRANSACTIONAL PRODUCER
// ========================================================================
final Properties producerProps = new Properties();
producerProps.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
producerProps.put(ProducerConfig.TRANSACTIONAL_ID_CONFIG, "myTransactionalId"); // Attiva le transazioni!
producerProps.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class.getName());
producerProps.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, StringSerializer.class.getName());

final KafkaProducer<String, String> producer = new KafkaProducer<>(producerProps);
producer.initTransactions(); // OBBLIGATORIO prima di iniziare qualsiasi transazione!

for (int i = 0; i < 100; i++) {
    final ProducerRecord<String, String> record = new ProducerRecord<>("topicA", "Key" + i, "Val" + i);

    producer.beginTransaction();
    producer.send(record);
    if (i % 2 == 0) {
        producer.commitTransaction(); // Conferma la transazione (i pari sono committati)
    } else {
        producer.flush();             // Scrive sul log prima di abortire (altrimenti resterebbe solo nel buffer locale)
        producer.abortTransaction();  // Annulla la transazione (i dispari sono abortiti)
    }
}
producer.close();

// ========================================================================
// 2. TRANSACTIONAL CONSUMER
// ========================================================================
final boolean readUncommitted = false; // false → "read_committed", true → "read_uncommitted"

final Properties consumerProps = new Properties();
consumerProps.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
consumerProps.put(ConsumerConfig.GROUP_ID_CONFIG, "groupA");
if (readUncommitted) {
    // Legge TUTTI i messaggi (sia pari committati sia dispari abortiti)
    consumerProps.put(ConsumerConfig.ISOLATION_LEVEL_CONFIG, "read_uncommitted");
} else {
    // Legge SOLO i messaggi appartenenti a transazioni committate (solo i pari)
    consumerProps.put(ConsumerConfig.ISOLATION_LEVEL_CONFIG, "read_committed");
}
consumerProps.put(ConsumerConfig.AUTO_OFFSET_RESET_CONFIG, "latest");
consumerProps.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());
consumerProps.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());

KafkaConsumer<String, String> consumer = new KafkaConsumer<>(consumerProps);
consumer.subscribe(Collections.singletonList("topicA"));
```

4.4 Atomic Forwarder: Consumer + Producer Combinati per End-to-End Exactly-Once (EOS)

(Rif: `AtomicForwarder.java`, `TransactionalForwarderTemplate.java`)

```
// ========================================================================
// 1. CONSUMER: legge solo messaggi committati e NON fa auto-commit
// ========================================================================
final Properties consumerProps = new Properties();
consumerProps.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
consumerProps.put(ConsumerConfig.GROUP_ID_CONFIG, "groupA");
consumerProps.put(ConsumerConfig.KEY_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());
consumerProps.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, StringDeserializer.class.getName());
consumerProps.put(ConsumerConfig.ISOLATION_LEVEL_CONFIG, "read_committed");
// Il consumer NON committa da solo: gli offset vengono salvati dentro la transazione del producer!
consumerProps.put(ConsumerConfig.ENABLE_AUTO_COMMIT_CONFIG, String.valueOf(false));

KafkaConsumer<String, String> consumer = new KafkaConsumer<>(consumerProps);
consumer.subscribe(Collections.singletonList("topicA"));

// ========================================================================
// 2. PRODUCER: transazionale + idempotente
// ========================================================================
final Properties producerProps = new Properties();
producerProps.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
producerProps.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class.getName());
producerProps.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, StringSerializer.class.getName());
producerProps.put(ProducerConfig.TRANSACTIONAL_ID_CONFIG, "forwarderTransactionalId");
producerProps.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG, String.valueOf(true));

final KafkaProducer<String, String> producer = new KafkaProducer<>(producerProps);
producer.initTransactions();

// ========================================================================
// 3. CICLO ATOMICO READ-PROCESS-WRITE
// ========================================================================
while (true) {
    final ConsumerRecords<String, String> records = consumer.poll(Duration.of(5, ChronoUnit.MINUTES));
    if (records.isEmpty()) {
        continue;
    }

    producer.beginTransaction();

    // A) Elabora i record letti e produce i risultati sul topic di destinazione
    for (final ConsumerRecord<String, String> record : records) {
        producer.send(new ProducerRecord<>("topicB", record.key(), record.value()));
    }

    // B) Calcola per ogni partizione letta il prossimo offset da leggere (lastOffset + 1)
    final Map<TopicPartition, OffsetAndMetadata> map = new HashMap<>();
    for (final TopicPartition partition : records.partitions()) {
        final List<ConsumerRecord<String, String>> partitionRecords = records.records(partition);
        final long lastOffset = partitionRecords.get(partitionRecords.size() - 1).offset();
        map.put(partition, new OffsetAndMetadata(lastOffset + 1));
    }

    // C) Salva atomicamente sia i messaggi prodotti su topicB sia l'avanzamento degli offset di groupA su topicA
    producer.sendOffsetsToTransaction(map, consumer.groupMetadata());
    producer.commitTransaction();
}
```